



Week 1: Foundations - Define concrete MVP scope: which clauses and which risks you want to

detect. - Set up project structure: backend (FastAPI), frontend, database. - Implement basic file upload and storage.

Week 2: Extraction & Storage - Implement text extraction and chunking from PDF/DOCX. - Design relational schema for contracts, clauses, and extracted fields. - Seed the system with a few sample contracts to develop against.

Week 3: Clause Classification & Risk Radar v1 - Implement simple heuristics and LLM calls to classify clauses into types. - Extract basic parameters (e.g., term length, auto-renew presence, liability cap amount). - Build the first version of the risk radar UI.

Week 4: Semantic Q&A - Add vector search for clauses. - Build the Q&A API using a RAG pattern. - Implement the Q&A UI and show clause references in answers.

Week 5: Comparison View - Implement upload of v1 + v2 contracts. - Align clauses using similarity and generate a "what changed" summary. - Create a side-by-side comparison UI.

Week 6: Polish and First User Tests - Improve prompts and clause grouping. - Add simple reminders for renewal dates. - Run the tool with real contracts from friendly users; collect feedback. set this up for me do deep reserach and then setit up

You can treat that 6-week plan as a real implementation spec. Below is a concrete setup: **exact clause/risk scope, tech stack, and week-by-week tasks** you can execute.

1. MVP scope: which clauses and risks to detect

For v1, lock onto **B2B SaaS subscription agreements + simple NDAs in English**. Focus on these clause categories and fields (you don't need everything from day one):

1. Term & Auto-renewal

- Fields: initial term length, renewal term, auto-renew presence, notice period (days), notice method (email / registered post).
- Risk patterns: evergreen term with short notice window, one-sided auto-renew, unclear notice method.^{[1] [2] [3]}

2. Termination

- Fields: termination for cause, termination for convenience, cure period (days), early termination fees, automatic termination events (bankruptcy, breach, etc.).
- Risk patterns: no termination for convenience, heavy early-termination penalties, very short cure period.^[4]

3. Payment & Price Changes

- Fields: fee structure, late payment penalties, interest rate, unilateral price increase rights, indexation.
- Risk patterns: uncapped unilateral price increase, very high late payment interest.

4. Liability & Indemnity

- Fields: liability cap (e.g., months of fees), excluded damages (indirect, consequential), indemnification obligations, carve-outs (IP infringement, data breaches).
- Risk patterns: **unlimited liability** or very high cap, broad indemnity obligations for the customer.^[4]

5. Renewal & Key Dates

- Fields: effective date, initial term end date, renewal dates, notice deadlines, review dates.
- CLM guides stress that tracking obligations, deliverables, and renewal dates is central to good contract management.^{[5] [6] [7]}

Those 5 buckets are enough for a serious MVP that already feels useful.

2. Tech stack and key libraries

You don't need anything exotic; use **boring, proven tools**:

- **Backend:** FastAPI (Python) for APIs + background tasks.
- **DB:** PostgreSQL with SQLAlchemy/Prisma for relational data (contracts, clauses, dates). CLM tools typically use a central database for obligations, dates, and metadata.^{[8] [7] [5]}
- **Vector DB:**
 - Simple option: Postgres + pgvector extension.
 - Or a dedicated store like Qdrant/Chroma (local to start).
- **Text extraction:**
 - **PDFs:** PyMuPDF (**fitz**) or PyPDF; recent evaluations highlight them as strong general-purpose PDF → text parsers.^{[9] [10]}
 - **DOCX:** python-docx.
 - **Structured tree:** optional but powerful – doc23 converts legal documents into hierarchical JSON trees, which is very helpful for clause-level analysis.^[11]
 - **Scanned PDFs:** pdf2image + pytesseract OCR, as recommended in modern OCR comparisons.^[12]
- **Frontend:** Next.js or Vite + React + Tailwind.
- **LLM layer:** Start with a hosted LLM (OpenAI/Anthropic/Claude, etc.) for classification and Q&A; you can abstract this behind a simple `llm_client.py`.

3. Detailed Week-by-Week execution plan

Week 1 – Foundations (MVP scope + skeleton)

Goal: Have clear scope and a working project skeleton with file upload + storage.

1. Lock scope and contract types

- Choose: *B2B SaaS subscription agreements + simple NDAs* only.
- Freeze v1 clause categories + fields (the 5 buckets above). Put them into a `config/risks.json` (`clause_type` → important fields → risk rules).

2. Project setup

- Monorepo structure:
 - `/backend` – FastAPI, DB models, extraction & AI services.
 - `/frontend` – Next.js/React app.
- Set up Postgres locally (Docker) and a basic migration setup.

3. File upload & storage

- FastAPI endpoint `POST /contracts/upload`:
 - Accepts PDF/DOCX file, contract type (`"saas"`, `"nda"`), and a name.
 - Stores file in a `contracts/` bucket or local folder.
 - Creates a `Contract` record with `status=UPLOADED`.
- Frontend: simple page with a file input + contract type dropdown.

If you finish early, set up basic auth (even email+password) so each user's contracts are separate.

Week 2 – Extraction & storage

Goal: Upload → extracted text & clause chunks saved in DB.

1. Implement extraction pipeline

- For PDFs: integrate PyMuPDF or PyPDF; test on real contracts to ensure headers, paragraphs, numbering survive reasonably. ^[13] ^[10]
- For DOCX: use `python-docx` to read paragraphs.
- Optional: add OCR fallback for scanned PDFs using `pdf2image` + `pytesseract`. ^[12]

2. Chunking and preliminary clause boundaries

- Split text into chunks based on:
 - Headings (`"1. TERM"`, `"2. TERMINATION"`), numbering patterns, and blank lines.
- Store each chunk as a `ClauseText` row linked to `Contract` (no classification yet).

3. Relational schema design

- `contracts` (`id`, `user_id`, `name`, `type`, `status`, `effective_date`, `initial_term_end`, `created_at`).

- `clauses` (`id`, `contract_id`, `raw_text`, `heading`, `section_number`, `clause_type` NULL, `risk_level` NULL, `metadata` JSONB).
- `key_dates` (`id`, `contract_id`, `label`, `date`, `source_clause_id`).
- Later you can add obligations, but don't over-design.

4. Seed with sample contracts

- Download a few **public** sample contracts or generate synthetic ones with an LLM and verify they are safe to use.
- Run the extraction pipeline end-to-end on these to shake out bugs.

Week 3 – Clause classification & Risk Radar v1

Goal: Classify clauses, extract basic parameters, and show risk radar UI.

1. Clause classification

- Step 1: simple keyword heuristics (e.g., “term”, “renew”, “termination”, “liability”, “indemnity”, “confidentiality”).
- Step 2: LLM classification:
 - For each clause chunk, send prompt like: *“Given this clause text, classify into one of [TERM, TERMINATION, PAYMENT, LIABILITY, DATA_PRIVACY, IP_CONFIDENTIALITY, OTHER] and give short label + rationale.”*
- Save `clause_type` and a short summary in `clauses.metadata`.

2. Parameter extraction

- For TERM/AUTO-RENEW: extract initial term (months/years), renewal period, notice period, notice method. ^{[2] [3] [1]}
- For TERMINATION: termination rights (for cause / convenience), cure period, early termination fees. ^[4]
- For LIABILITY: cap (e.g., “12 months of fees” → numeric), excluded damages, carve-outs.
- Use LLM with structured output (JSON) and validate before saving.

3. Risk rules engine v1

- Implement simple Python rules:
 - If `auto_renew == true` and `notice_days < 60` → mark as “Medium/High risk”.
 - If `liability_cap` is NULL or very large → “High risk”.
 - If no termination for convenience → “Medium risk”.
- Store `risk_level` and `risk_reason` in `clauses`.

4. Risk Radar UI v1

- Frontend page: `/contracts/:id/risk` with:
 - Overall risk summary (e.g., highest risk level across clauses).

- Sections by category (Term & Renewal, Termination, Payment, Liability, Data/IP) listing clauses with risk badges.

- Clicking a clause shows raw text + summary + risk explanation.

By the end of Week 3, your product already “feels” like something real.

Week 4 – Semantic Q&A (RAG)

Goal: Ask natural questions and get answers with clause references.

1. Embeddings & vector search

- For each `clause`, create an embedding and store it in pgvector or a vector DB.
- Ensure index by `contract_id` so search is scoped to one contract.

2. Q&A API

- Endpoint POST `/contracts/{id}/qa` with question.
- Pipeline:
 - Embed question.
 - Retrieve top K clauses for that contract.
 - Call LLM with system prompt: *“Answer ONLY from these clauses; cite clause headings/section numbers; use simple language.”*
- Return answer + list of clause ids used.

3. Q&A UI

- Chat-like component on `/contracts/:id/qa`.
- Messages show AI answer plus clickable clause references (jump to the clause in the risk radar view).

4. Guardrails

- If retrieval returns nothing relevant, respond with “I cannot find anything in this contract about X” rather than hallucinating.

Week 5 – Comparison view (“what changed?”)

Goal: Upload old + new contract and show differences.

1. Two-contract workflow

- Frontend: screen to upload “old” and “new” contracts (or select existing ones).
- Backend: extraction/classification pipeline runs on both if not already processed.

2. Clause alignment

- For each clause in v2, find most similar clause in v1 via embedding similarity (same contract type).
- Classify pairs into:

- Unchanged / minor wording change
 - New (no close match)
 - Removed (v1 clause with no match in v2)
 - Focus on **high-impact clause types first**: Term, Termination, Liability, Payment. ^[4]
- 3. Diff summarization**
- For changed pairs, ask LLM to summarize: “In v2, liability cap changed from X to Y; renewal changed from manual to auto-renew, etc.”
 - Generate overall “what changed” summary.
- 4. Side-by-side UI**
- For each clause pair, show v1 text vs v2 text with change badges.
 - Top panel: bullet list of key changes (“Added auto-renew; liability cap doubled; notice period reduced from 60 to 30 days”).

Week 6 – Polish, reminders, and user tests

Goal: Make it nicer to use and get real feedback.

- 1. Prompt + grouping improvements**
- Tweak classification and risk prompts based on contracts that mis-classified.
 - Refine risk rules to reduce false alarms.
- 2. Renewal & notice reminders**
- From TERM and RENEWAL clauses, extract:
 - contract end date, renewal pattern, non-renewal notice deadline.
 - Create `key_dates` records; CLM practice emphasizes tracking renewal and obligation dates for compliance and revenue protection. ^{[6] [7] [5]}
 - Implement a small scheduler (e.g., cron + background job) to send email notifications X days before key dates.
- 3. UX polish**
- Make risk radar easy to scan on one screen.
 - Show a “confidence” note and legal disclaimer on every view.
- 4. First user tests**
- Find 3–5 friendly users (agency owners, freelancers, SaaS founders).
 - Sit on a call, upload 1–2 of their contracts, and watch them use the tool.
 - Note where they get confused and what questions they actually ask.

If you want, next step I can:

- Turn this into a GitHub issue list (with labels per week) that you can drop into a repo, and

- Help you design the exact DB schema and a first version of the FastAPI endpoints (path + request/response shapes) so you can start coding today.

✱✱

1. <https://olenderfeldman.com/the-pitfalls-of-autorenewal-contracts/>
2. <https://marell-lawfirm.com/the-pesky-auto-renewal-clause/>
3. <https://www.tahardinglawfirm.com/blog/what-to-watch-for-in-auto-renewal-clauses-during-contract-review/>
4. <https://www.floreslegalpllc.com/termination-and-renewal-clauses-strategic-considerations-for-supply-and-distribution-agreements/>
5. <https://www.cobblestonesoftware.com/blog/the-8-stages-of-the-contract-lifecycle>
6. <https://www.contractsafe.com/blog/stages-contract-management>
7. <https://www.jaggaer.com/blog/what-is-clm-contract-lifecycle-management>
8. <https://www.salesforce.com/in/sales/revenue-lifecycle-management/what-is-contract-lifecycle-management/>
9. <https://www.goodnovel.com/qa/best-python-library-pdf-text-extraction>
10. <https://unstract.com/blog/evaluating-python-pdf-to-text-libraries/>
11. <https://pypi.org/project/doc23/>
12. <https://ploomber.io/blog/pdf-ocr/>
13. <https://medium.com/@hchenna/technical-comparison-python-libraries-for-document-parsing-318d2c89c44e>
14. <https://www.nge.com/news-insights/publication/contract-auto-renewals-not-necessarily-so-automatic-as-recent-ny-case-demonstrates/>
15. <https://www.sirion.ai/library/contract-management/contract-lifecycle-management-process-stages-steps/>